



Universidad de Santiago de Chile
Facultad de Ciencias
Departamento de Física
Astrofísica con mención en Ciencia de Datos

Proyecto Sonda Parker Solar Probe

Informe de Avance – Sprint 2

*Martina Molina Rizzo, Thiare Molina Tapia
Bárbara Quilodrán Hijerra, Ivannia Villalba Berrios*

Asignatura: Desarrollo de Software — **Profesor:** Ricardo Hasbun
26 de julio de 2026

Índice

1. Introducción	2
2. Funciones a Priorizar	2
2.1. descargar_data.py	2
2.2. Filtrar_data.py	2
3. Uso de Ramas, Commits y Colaboración	3
3.1. Estructura de Ramas	3
3.2. Convención de Commits y Colaboración	3
4. Pruebas Unitarias	3
4.1. Pruebas a descargar_data.py	3
4.1.1. test_listar_archivos_exito(mocker)	3
4.1.2. test_listar_archivos_error_servidor(mocker)	3
4.1.3. test_listar_archivos_sin_cdfs(mocker)	4
4.1.4. test_descargar_archivo_exito(mocker)	4
4.1.5. test_descargar_archivo_invalido(mocker)	4
4.2. Pruebas a Filtrar_data.py	4
4.2.1. test_filtrar_lista_vacia()	4
4.2.2. test_filtrar_elimina_nulos(datos_prueba)	4
4.2.3. test_filtrar_por_rango(datos_prueba)	4
4.2.4. test_filtrar_mantiene_estructura(datos_prueba)	4
4.2.5. test_filtrar_entrada_invalida()	4
5. GitHub Actions	4
6. Actualización del README.md	5
7. Actualización del Product Backlog y Sprint Backlog	5
7.1. Product Backlog Actualizado	5
7.1.1. Historias de Usuario Iniciales	5
7.1.2. Nuevas Historias de Usuario Incorporadas (Sprint 2)	6
7.2. Sprint Backlog del Sprint 2	6
8. Registro de Cambios respecto al Sprint 1	7
9. Demo Parcial del Sistema	8
10. Retroalimentación del Cliente	8

1. Introducción

El presente documento detalla el trabajo realizado y los objetivos alcanzados durante el Sprint 2 del proyecto. En esta etapa, nuestro equipo ha transitado desde la fase de planificación y configuración inicial hacia la implementación práctica de soluciones, consolidando al mismo tiempo un flujo de trabajo óptimo y sólido para completar todo lo prometido en el Sprint 1.

A lo largo de las siguientes secciones, se expondrá el avance en la implementación de las funcionalidades priorizadas del sistema, respaldadas por un control de versiones estructurado mediante el uso de ramas y commits en GitHub. Con el objetivo de asegurar la calidad y estabilidad del código, se detallará la integración de pruebas unitarias utilizando la herramienta Pytest [8], así como los primeros pasos en la automatización de procesos mediante GitHub Actions [9].

Adicionalmente, el informe aborda la gestión continua del proyecto, incluyendo la actualización del backlog y de la documentación técnica (`README.md`) con instrucciones claras de ejecución, y un registro de los cambios aplicados en comparación con el Sprint 1. El documento concluye con la evidencia de la demostración parcial del sistema, sentando las bases operativas para el próximo ciclo de desarrollo.

2. Funciones a Priorizar

Nuestra mayor prioridad para este sprint fue la implementación de las funciones indispensables para la ejecución del motor de backend en Python, el cual extrae, procesa y prepara los datos científicos que posteriormente alimentarán la plataforma web. Nuestro proyecto cuenta con dos scripts principales: `descargar_data.py` y `Filtrar_data.py`.

2.1. `descargar_data.py`

Este script es el primer componente ejecutable del sistema. Se encarga de conectar con los repositorios de la NASA SPDF/CDAWeb [1, 5] para extraer los archivos binarios científicos en formato `.cdf` mediante las funciones `listar_archivos_del_año()` y `descargar()`, apoyándose en la librería `BeautifulSoup4` [11]. Adicionalmente, cuenta con la función `espacio_libre_gb()`, la cual utiliza la librería `shutil` para verificar el almacenamiento disponible en disco previo a la descarga, previniendo la saturación del sistema local.

2.2. `Filtrar_data.py`

Este script procesa los datos científicos extraídos. Por medio de iteraciones, clasifica las variables en estructuras que, mediante la librería `Pandas` [10], se convierten en `DataFrames`. Tras limpiar el ruido instrumental (eliminando lecturas físicas no válidas como -1×10^{30}), calcular la distancia al Sol y detectar los perihelios con su número de órbita correspondiente [6], ejecuta la función `preparar_datos_para_web()` para realizar el remuestreo temporal a intervalos de 1 hora e interpolación lineal de datos faltantes.

Aceptando la retroalimentación del Sprint 1, reconocemos que la latencia y la tasa de transferencia de la API/servidores HTTP de la NASA son factores externos fuera del control de nuestro código. Por ello, la estrategia de arquitectura de este módulo busca **desacoplar el flujo de datos**. Al procesar localmente los archivos `.cdf` y generar una

base de datos reducida en formato `psp_datos_filtrados.json`, se garantiza que, cuando la interfaz web sea integrada en el Sprint 3, consume datos precalculados de respuesta inmediata, cumpliendo nuestro Requerimiento No Funcional de rendimiento.

3. Uso de Ramas, Commits y Colaboración

3.1. Estructura de Ramas

Para mantener un flujo de desarrollo ordenado, se utilizó la siguiente estructura de ramificación:

- **main**: Contiene únicamente versiones estables, probadas y listas para entrega (código de producción).
- **feature/descarga-data**: Desarrollo y desacoplamiento del módulo de descarga y verificación de espacio en disco.
- **feature/filtrar-data**: Implementación de la lógica de limpieza, interpolación y detección de perihelios con Pandas [10].
- **feature/pytest-suite**: Creación y mockeo de las pruebas unitarias [8].
- **feature/frontend-index**: Maquetado HTML y lógica en JavaScript para la manipulación del DOM(Document Object Model) y exportación de archivos.

3.2. Convención de Commits y Colaboración

Se aplicaron estándares de *Conventional Commits* (`feat:`, `docs:`, `test:`) para garantizar la trazabilidad del historial. La integración de código hacia la rama **main** se realizó exclusivamente mediante *Pull Requests* (PR), los cuales requirieron la revisión cruzada de al menos un integrante del equipo y la aprobación automatizada del pipeline de pruebas antes de ejecutarse el *merge*.

4. Pruebas Unitarias

Implementamos pruebas unitarias a los dos módulos principales: `descargar_data.py` (5 pruebas para validar funciones de descarga y listado sin depender del servidor real) y `Filtrar_data.py` (5 pruebas para comprobar limpieza de nulos, filtros por valor y formato de datos) [8].

4.1. Pruebas a `descargar_data.py`

4.1.1. `test_listar_archivos_exito(mock)`

Comprueba que cuando el servidor responde correctamente (código HTTP 200) y entregamos contenido HTML con enlaces a archivos `.cdf`, la función identifica y retorna la lista esperada [11].

4.1.2. `test_listar_archivos_error_servidor(mock)`

Evalúa qué pasa si el servidor web de la NASA está caído o se interrumpe la conexión.

4.1.3. `test_listar_archivos_sin_cdfs(mock)`

Valida el caso donde la web responde correctamente, pero no hay archivos de extensión `.cdf` disponibles.

4.1.4. `test_descargar_archivo_exito(mock)`

Comprueba el proceso de guardar el archivo descargado en el almacenamiento local.

4.1.5. `test_descargar_archivo_invalido(mock)`

Simula la solicitud de un archivo que fue removido o renombrado en el servidor origen (Respuesta HTTP 404).

4.2. Pruebas a `Filtrar_data.py`

4.2.1. `test_filtrar_lista_vacia()`

Valida que si el módulo recibe una lista vacía `[]`, devuelva otra lista vacía inmediatamente.

4.2.2. `test_filtrar_elimina_nulos(datos_prueba)`

Revisa la eliminación de registros que contengan mediciones nulas o corruptas (-1×10^{30}) [10].

4.2.3. `test_filtrar_por_rango(datos_prueba)`

Verifica que al aplicar un parámetro numérico (ej. `min_val = 15`) el resultado solo incluya valores que cumplan con la condición.

4.2.4. `test_filtrar_mantiene_estructura(datos_prueba)`

Comprueba que el proceso de filtrado conserve la estructura de llaves (fecha, valor, etc.) de cada objeto.

4.2.5. `test_filtrar_entrada_invalida()`

Evalúa la robustez del código enviando `None` en lugar de una lista.

5. GitHub Actions

Para garantizar la calidad del proyecto software y poder automatizar la validación de los scripts, se implementó un flujo de Integración Continua utilizando GitHub Actions [9]. Este pipeline permite realizar las pruebas de manera automática ante cada modificación del código:

- **Desencadenadores (Triggers):** El pipeline se ejecuta automáticamente ante cada evento de `push` o `pull_request` dirigido a las ramas `main` y `develop`.

- **Ambiente de Ejecución:** Se configura un entorno virtual sobre `ubuntu-latest` con Python 3.10.
- **Pasos del Pipeline:**
 1. Instalación de dependencias del proyecto descritas en `requirements.txt` (`pandas`, `cdflib`, `requests`, `beautifulsoup4`, `pytest`, `pytest-mock`).
 2. Verificación de sintaxis y estándares de código.
 3. Ejecución automatizada de la suite completa de 10 pruebas unitarias mediante el comando `pytest` [8].

6. Actualización del README.md

Se realizó una actualización del archivo `README.md` del repositorio para añadir las actividades ejecutadas en el Sprint 2:

- Automatización de la descarga de archivos `.cdf` desde los servidores de la NASA mediante la función `descargar_data.py`, añadiendo las alertas por espacio insuficiente en disco y el control para evitar descargas duplicadas.
- Procesamiento y análisis de datos a través de la función `Filtrar_data.py`, organizando variables (densidad, velocidad, temperatura) y determinando la órbita mediante los perihelios de la Parker Solar Probe [6].
- Instrucciones para la ejecución local de las pruebas unitarias con `pytest` [8].

7. Actualización del Product Backlog y Sprint Backlog

Tras la retroalimentación de la entrega del Sprint 1, el equipo reestructuró la planificación para reflejar con mayor precisión las restricciones de infraestructura, el desacoplamiento de la arquitectura, las pruebas automatizadas y la experiencia de la cliente.

7.1. Product Backlog Actualizado

El *Product Backlog* representa la lista global y priorizada de todas las Historias de Usuario (HU) del proyecto:

7.1.1. Historias de Usuario Iniciales

- **HU1 – Exportación de datos:** Como investigadora, quiero presionar un botón para exportar la tabla procesada en formatos accesibles como `.txt` [4].
- **HU2 – Filtrado temporal:** Como investigadora, quiero filtrar los datos de la sonda por un intervalo de fechas específico.
- **HU3 – Trazabilidad de la fuente:** Como usuario, quiero ver la fuente oficial de los datos (NASA SPDF/CDAWeb) visible en la interfaz para respaldar la validez científica [1].

- **HU4 – Conversión de unidades:** Como usuario, quiero convertir dinámicamente las distancias entre Radios Solares (R_s), Unidades Astronómicas (AU) y Radios Terrestres (R_e).

7.1.2. Nuevas Historias de Usuario Incorporadas (Sprint 2)

- **HU5 – Estructuración de datos:** Como investigadora, quiero que los datos crudos se limpien y estructuren en un DataFrame de Pandas (Órbita, fecha, hora, posición RTN, densidad, temperatura y velocidad) [10].
- **HU6 – Conexión y descarga automatizada:** Como investigadora, quiero conectar el sistema a NASA SPDF/CDAWeb para descargar automáticamente archivos `.cdf` de perihelios [5].
- **HU7 – Visualización tabular:** Como usuario, quiero ver los datos filtrados en una tabla interactiva en la web.
- **HU8 – Tiempo de respuesta (RNF):** Como usuario, quiero que la consulta en la interfaz web responda en menos de 7 segundos.
- **HU9 – Gestión de almacenamiento en disco:** Como usuario, quiero recibir una advertencia antes de iniciar la descarga si mi almacenamiento libre es menor a 5 GB.
- **HU10 – Desacoplamiento y arquitectura liviana:** Como investigadora, quiero que el procesamiento pesado de archivos `.cdf` se realice en segundo plano, para que la interfaz web sea liviana.
- **HU11 – Calidad de código y reproducibilidad:** Como desarrollador, quiero que las funciones cuenten con pruebas unitarias mockeadas en Pytest [8].

7.2. Sprint Backlog del Sprint 2

El *Sprint Backlog* detalla el conjunto de tareas técnicas ejecutadas y planificadas durante este ciclo:

1. **TSK-01 — Módulo de descarga automática desde la NASA (HU6):** Desarrollo del script `descargar_data.py` para la conexión HTTP y extracción de archivos `.cdf` desde SPDF/CDAWeb [5, 7].
Estado: Completado
2. **TSK-02 — Verificación de almacenamiento local (HU9):** Implementación del control de espacio libre en disco previo a la descarga mediante la librería `shutil` [3].
Estado: Completado
3. **TSK-03 — Módulo de limpieza e interpolación en Pandas (HU2, HU5, HU10):** Desarrollo del script `Filtrar_data.py` para la estructuración de la matriz de datos, remuestreo a 1 hora e interpolación lineal de datos faltantes [10].
Estado: Completado

4. **TSK-04 — Algoritmo de detección de perihelios (HU2, HU5):** Diseño del algoritmo de búsqueda de mínimos locales de distancia al Sol en ventanas de 15 días y etiquetado automático de órbitas [6].
Estado: Completado
5. **TSK-05 — Suite de pruebas unitarias con Pytest (HU11):** Construcción de 10 pruebas unitarias con `pytest` y simulaciones de respuesta HTTP (`pytest-mock`) [8].
Estado: Completado
6. **TSK-06 — Pipeline de Integración Continua CI/CD (HU11):** Configuración del flujo automatizado mediante GitHub Actions (`.github/workflows/pytest.yml`) [9].
Estado: Completado
7. **TSK-07 — Interfaz web e integración frontend (HU3, HU4, HU7, HU8):** Diseño del maquetado estático y lógica en JavaScript para lectura del JSON local, conversión de unidades y tabla interactiva.
Estado: Pendiente (Planificado para Sprint 3)
8. **TSK-08 — Módulo de exportación de archivos (HU1):** Implementación del botón de descarga en formato `.txt` estructurado mediante objetos Web Blob.
Estado: Pendiente (Planificado para Sprint 3)
9. **TSK-09 — Despliegue en servidor de producción (HU3, HU8):** Configuración del despliegue público de la plataforma web en GitHub Pages / Vercel.
Estado: Pendiente (Planificado para Sprint 3)

8. Registro de Cambios respecto al Sprint 1

En comparación con la etapa inicial de diseño del Sprint 1, los principales avances técnicos consolidados en el backend durante este Sprint 2 son:

- **Conexión y extracción automatizada:** Se logró la lectura y descarga programática de los archivos binarios científicos `.cdf` desde la NASA utilizando la librería `cdflib` [7] y parseo de rutas con `BeautifulSoup4` [11].
- **Control preventivo de infraestructura:** Se incorporó la validación de almacenamiento disponible con `shutil` para prevenir interrupciones o llenado de disco local durante la descarga masiva.
- **Procesamiento e interpolación de series temporales:** Se implementó la unificación de variables de plasma (densidad, velocidad y temperatura) y trayectoria orbital con `Pandas`, aplicando remuestreo a 1 hora e interpolación lineal para mitigar datos faltantes [10].
- **Algoritmo propio de perihelios:** Se diseñó la lógica matemática de ventanas deslizantes de 15 días para la asignación automatizada del número de órbita oficial [6].

- **Estructuración de artefacto liviano:** Se definió el esquema de exportación a `psp_datos_filtrados.json`, sentando las bases para el acople desacoplado con la futura interfaz web.
- **Aseguramiento de calidad (Testing y CI/CD):** Se incorporaron 10 pruebas unitarias con `pytest` y se automatizó la integración continua mediante GitHub Actions [8, 9].

9. Demo Parcial del Sistema

En esta etapa nos hemos centrado en desarrollar y validar la arquitectura base de nuestro backend. La demostración parcial del Sprint 2 está conformada por la ejecución secuencial e integración funcional de los dos módulos de Python en entorno local:

1. **Ejecución del Módulo de Descarga (`descargar_data.py`):** Al ejecutar el script desde la terminal, el sistema verifica primero que el espacio libre en disco sea superior a 2 GB mediante `shutil`. Posteriormente, explora los directorios HTTP de la NASA, identifica los archivos binarios `.cdf` requeridos y realiza la descarga progresiva omitiendo los registros ya existentes en el equipo local [5].
2. **Ejecución del Módulo de Procesamiento (`Filtrar_data.py`):** A continuación, se ejecuta el script de filtrado, el cual toma como insumo los binarios descargados. El programa limpia las mediciones corruptas, detecta los mínimos de distancia (perihelios), remuestra la información a intervalos de 1 hora, interpola los huecos de transmisión y exporta la base de datos procesada en el archivo estructurado `psp_datos_filtrados.json` [10].
3. **Verificación de Salida y Preparación Frontend:** La validación de esta demo parcial se realizó a nivel de consola e inspección del archivo JSON generado, confirmando que la matriz de datos contiene todas las variables científicas necesarias y la numeración de órbitas correcta. Este archivo servirá como insumo directo para la construcción e interacción de la interfaz web en el Sprint 3.

10. Retroalimentación del Cliente

Tras presentar la demostración parcial del procesamiento de datos en Python a la investigadora Matilde Coello [4], se obtuvieron los siguientes comentarios que orientarán la entrega final:

- **Evaluación de los datos procesados:** Tras la revisión de las primeras muestras, la cliente nos felicitó por haber implementado el método de interpolación lineal que ella nos había sugerido, dentro del script `Filtrar_data.py`. Debido al elevado volumen y la alta frecuencia de muestreo de las mediciones de la sonda, existen breves vacíos temporales causados por pausas instrumentales o de transmisión. La implementación de la interpolación permite corregir estas discontinuidades y estructurar una serie temporal uniforme a intervalos de 1 hora, garantizando la consistencia de los datos para sus análisis astrofísicos.
- **Requerimientos acordados para el Sprint 3 (Interfaz Web):**

1. Construir la interfaz gráfica (`index.html`) que consuma directamente el archivo JSON generado.
2. Permitir la conversión dinámica de distancias a Unidades Astronómicas (AU) en la pantalla.
3. Incluir la cabecera con las unidades de medida en el archivo `.txt` que exporte la aplicación.

Referencias

- [1] National Aeronautics and Space Administration [NASA]. (s.f.). *Coordinated Data Analysis Web (CDAWeb) - Space Physics Data Facility (SPDF)*. <https://cdaweb.gsfc.nasa.gov/>
- [2] Google. (2026). *Gemini* (Versión 2026) [Modelo de lenguaje grande]. <https://gemini.google.com/>
- [3] National Aeronautics and Space Administration [NASA]. (s.f.). *Parker Solar Probe*. NASA Science. <https://science.nasa.gov/mission/parker-solar-probe/>
- [4] Universidad de Santiago de Chile [USACH]. (s.f.). *Matilde Coello*. Departamento de Física. <https://fisica.usach.cl/es/matilde-coello>
- [5] National Aeronautics and Space Administration [NASA]. (s.f.). *Space Physics Data Facility (SPDF)*. <https://spdf.gsfc.nasa.gov/pub/data/>
- [6] Johns Hopkins Applied Physics Laboratory. (s.f.). *The mission - Parker Solar Probe: Spacecraft and Instruments*. <https://parkersolarprobe.jhuapl.edu/>
- [7] Maffeo, M., & Harter, B. (2024). *cdflib: A Python module to read/write Common Data Format (CDF) files*. Python Package Index (PyPI). <https://pypi.org/project/cdflib/>
- [8] Krekel, H., Oliveira, B., Pfannschmidt, F., Bruynooghe, F., Laughner, T., & Schlammmer, F. (2024). *pytest: helps you write better programs* (Versión 8.x) [Software framework]. <https://docs.pytest.org/>
- [9] GitHub. (2024). *GitHub Actions Documentation: Automating your workflow with CI/CD*. GitHub Docs. <https://docs.github.com/en/actions>
- [10] McKinney, W., & The pandas development team. (2024). *pandas-dev/pandas: pandas* (Versión 2.x) [Software de análisis de datos]. Zenodo. <https://pandas.pydata.org/>
- [11] Richardson, L. (2024). *Beautiful Soup Documentation* (Versión 4.x). Crummy. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>